

Vision Document
Master Thesis Proposal

Title: Integrating Open Large Language Models into the Jupyter Notebook Reproducibility Pipeline

Supervisor: Dr. Sheeba Samuel
Student: Erisa Zaimi
Program: M.Sc. Web Engineering
University: TU Chemnitz
Date: April 2026

1. Introduction

Jupyter notebooks have become one of the most widely used tools for sharing and documenting computational research. They allow researchers to combine code, explanations, and results in a single interactive document, making them popular across many scientific fields. However, reproducing the results of a notebook is often harder than it looks. When a notebook is run in a different environment than the one it was originally written in, missing or incompatible library versions frequently cause it to break. This means that a large portion of published notebooks cannot be reliably re-executed, which is a significant problem for open and reproducible science. The FAIR Jupyter project has investigated this issue at scale by building a pipeline that assesses the reproducibility of Jupyter notebooks from biomedical publications. While this work has made important progress in detecting and documenting failures, it does not yet attempt to fix them.

This thesis aims to take the next step by integrating open Large Language Models (LLMs) into the FAIR Jupyter pipeline to automatically explain and repair dependency-related errors. When a notebook fails to execute, the system will generate a human-readable explanation of the error and suggest a fix, which will then be applied and validated by re-running the notebook. By using open, locally deployable models, the system also stays in line with the open science principles that the FAIR Jupyter project is built on. Rather than collecting and executing notebooks from scratch, this thesis builds directly on the existing dataset made available by the FAIR Jupyter project, which contains thousands of notebooks from biomedical publications along with their execution results, dependency metadata, and error logs [3]. This provides a realistic and structured foundation for developing and evaluating the proposed repair system.

2. Problem Statement

Computational reproducibility is a fundamental requirement of trustworthy science, meaning that a published analysis should be re-executable by others and produce the same results. In the context of Jupyter notebooks, this expectation is very often not met. Pimentel et al. [4] conducted a large-scale study collecting 1.4 million notebooks from GitHub and attempting to execute a subset of 863,878 of them, finding that only 24.11% ran without errors and only 4.03% produced the same results as originally reported. Samuel and Mietchen [2] confirmed this trend specifically for biomedical publications, finding similarly low reproducibility rates among notebooks associated with peer-reviewed articles. These numbers make it clear that this is not a minor or occasional issue, but it is a widespread and systematic problem.

A major cause of these failures is dependency-related errors. Researchers often do not pin the exact versions of the libraries they use, which means that as the Python ecosystem evolves over time, previously working notebooks start to break when executed in a different environment. The error messages produced in these situations are often cryptic and difficult to interpret, especially for users with limited programming experience. This creates a situation where a

large portion of published scientific work is technically inaccessible to the very people it is meant to reach.

While some efforts have been made to detect and document these failures at scale, the problem of automatically explaining and repairing them remains largely unsolved. This thesis argues that integrating open Large Language Models into the existing FAIR Jupyter reproducibility pipeline offers a promising way to close this gap.

3. State of the Art

This section reviews the existing work most relevant to this thesis across four areas: tools for assessing and restoring Jupyter notebook reproducibility, approaches to automated dependency resolution, the use of LLMs for debugging and program repair, and broader surveys of LLM-based automated program repair. It also discusses where the gaps are and what this thesis contributes on top of the existing work.

3.1 Measuring Reproducibility of Jupyter Notebooks

The reproducibility problem in Jupyter notebooks has been studied from several different angles. Pimentel et al. [4] carried out one of the largest studies in this area, collecting 1.4 million notebooks from GitHub and attempting to execute 863,878 of them, showing that the vast majority failed due to issues such as missing dependencies, incorrect execution order, and undocumented environment requirements. Samuel and Mietchen [2] extended this analysis to the biomedical domain, finding similarly low reproducibility rates among notebooks published alongside peer-reviewed articles. Samuel and König-Ries [6] introduced ReproduceMeGit, a visualization tool that lets repository owners and users check the reproducibility of any GitHub repository containing Jupyter notebooks. It runs the notebooks and gives a clear summary of which ones executed successfully, which ones crashed with errors, and which ones produced different outputs from the originals. While this was a useful step forward, ReproduceMeGit is focused on showing users what happened — it does not try to explain why it happened or fix it.

3.2 Restoring Reproducibility of Jupyter Notebooks

Beyond measuring failures, a separate line of work focuses on restoring notebook executability. Wang et al. [11] studied the problem of assessing and restoring reproducibility by analyzing how execution environments could be reconstructed, laying the groundwork for automated environment inference. Building on this, Wang et al. [10] introduced SnifferDog, a tool that tries to automatically figure out which Python packages a notebook needs in order to run. It does this by statically analyzing the notebook’s code and matching it against a database of known package APIs and versions. In their evaluation, SnifferDog successfully inferred the correct execution environment for 92.6% of notebooks tested, and 90.2% of those could then be executed automatically. This makes an important point that is relevant to this thesis: many notebooks that look broken are not fundamentally broken — they simply lack the right

dependency information. More recently, Samuel et al. [1] extended the FAIR Jupyter pipeline with an automated containerization approach that uses Docker to reconstruct the execution environment of a repository before running its notebooks. Their results show that many notebooks previously considered broken due to dependency conflicts can actually be restored through automated environment reconstruction, suggesting that a large part of the reproducibility problem is really an environment management problem rather than a fundamental flaw in the research code itself. However, even this more advanced pipeline stops at the point of execution — when a notebook still fails after containerization, the system simply logs the error without explaining what went wrong or attempting to fix it. This is exactly the gap this thesis targets.

3.3 Automated Dependency Resolution

Several tools have been developed to address the problem of missing or incompatible Python dependencies. As discussed above, SnifferDog [10] takes a static analysis approach to infer required packages, but it cannot always determine the exact version needed, especially when package APIs have changed across versions. Bartlett et al. [5] addressed this limitation by using LLMs directly for dependency resolution. Their system, PLLM, uses retrieval-augmented generation (RAG) to iteratively infer missing or incorrect dependencies by observing execution feedback and parsing error messages in natural language. In their experiments, PLLM outperformed knowledge-graph-based baselines by +15.97% over ReadPyE and +21.58% over PyEGo, with particularly strong results for notebooks involving complex machine learning libraries. This thesis explores the possibility of adapting PLLM’s RAG-based design into the FAIR Jupyter pipeline as part of the automated repair layer. More specifically, since PLLM demonstrated that grounding repair suggestions in live package version data from PyPI consistently improves fix rates, this thesis incorporates PyPI-grounded RAG into Step 2 of the proposed approach — fetching live package version information at repair time to produce more precise and up-to-date fix suggestions. This differentiates the approach from naive prompting and has strong precedent in the literature.

Byam [14] further demonstrates the value of LLM-based repair for breaking dependency updates in Java projects, using build process diagnostics and breaking dependency analysis to guide the LLM in updating client code. Their multi-granularity evaluation — at the build, file, and compilation error levels — provides a useful methodological reference for structuring the evaluation in this thesis.

3.4 LLM-Based Debugging and Repair

Beyond dependency resolution, LLMs have been explored more broadly for automated program repair. Yang et al. [15] provide a comprehensive taxonomy of 63 LLM-based APR systems published between 2022 and 2025, categorizing them into four paradigms: fine-tuning, prompting, procedural pipelines, and agentic frameworks. Their analysis highlights that agentic frameworks are well suited for multi-hunk or cross-file bugs but come at the price of increased latency and complexity — a trade-off that directly informs the design decision in this thesis to adopt a single-round procedural repair approach rather than a full multi-round agentic loop.

The survey also confirms that retrieval-augmented contexts strengthen repair quality across all paradigms, further motivating the RAG component described in Section 3.3.

Tang et al. [16] present SynFix, a dependency-aware program repair approach that constructs a RelationGraph to model structural relationships across a codebase and uses it to synchronize repairs across interdependent components. Evaluated on SWE-bench, SynFix resolves 52.33% of issues in SWE-bench Lite and outperforms several established baselines. While SynFix targets general software bugs rather than notebook dependency errors specifically, its use of structured dependency context as input to the LLM is relevant to the design of the repair prompt in this thesis.

Xia and Zhang [12] proposed ChatRepair, a conversation-driven approach that generates code patches and refines them through an iterative feedback loop based on test failures and previous repair attempts. The key idea — that re-execution feedback can guide the next repair step — is directly relevant to this thesis and informs how the repair loop is structured: after a fix is applied and the notebook is re-executed, the outcome is recorded and can be used to refine the repair strategy.

More specifically related to notebooks, Grotov et al. [7] proposed an approach where an LLM agent iteratively attempts to fix errors in computational notebooks, and also released a dataset of buggy notebooks to support further research. This dataset could be useful in the evaluation phase of this thesis, particularly to cover error types that may not appear frequently enough in the FAIR Jupyter biomedical dataset. Grotov et al. [8] built on this by showing that an LLM agent can look at the error output after each execution attempt, decide what to change, apply the fix, and try again. This feedback-driven process is relevant to the repair approach proposed in this thesis. A full multi-round agent setup is not adopted here because, as the survey by Yang et al. [15] confirms, agentic frameworks introduce significantly higher latency and complexity — costs that scale poorly when processing large numbers of notebooks in batch. To assess the impact of this design decision, an ablation study comparing one-round versus two-round repair will be included in the evaluation, measuring the trade-off between additional computational time and repair success rate.

On the model selection side, Szalontai et al. [9] found that smaller models specifically trained on code, such as CodeLlama, tend to perform better than larger general-purpose models when it comes to fixing code errors. They also found that how the prompt is written has a significant impact on the quality of the repair. Both of these findings are useful for deciding which open LLM to use in this thesis and how to design the prompts for the explanation and repair steps.

It is also worth noting that LLM-based error explanation has been explored in other areas outside of notebook repair. For example, in DevOps, LLMs have been used to help engineers understand system logs and identify what went wrong in a running service. In software development, tools like GitHub Copilot and similar IDE assistants can explain error messages and suggest fixes directly inside the code editor. However, these tools are built for situations where a developer is actively working and manually asks for help. They are not designed to

run automatically across hundreds of notebooks in batch, and they have no connection to reproducibility pipelines or the kind of structured metadata that FAIR Jupyter provides. For these reasons, they are not directly applicable to the problem this thesis addresses.

3.5 Gap and Contribution of This Thesis

Taken together, the existing work shows that the individual building blocks are already available: pipelines exist for detecting reproducibility failures [1], [2], [3], tools exist for inferring and restoring dependencies [5], [10], and LLM-based agents have been shown to fix errors in notebooks iteratively [7], [8]. It is also worth noting that LLM-based tools for explaining and fixing code errors in notebooks already exist in other contexts. Jupyter AI [13], for example, is an open-source extension that allows users to interactively ask an LLM to explain code, fix errors, and generate new cells directly within JupyterLab. However, tools like Jupyter AI are designed as interactive assistants for individual users — they require a user to manually trigger an explanation or fix after an error occurs. They are not designed to operate autonomously within a reproducibility pipeline, where notebooks are executed in batch, failures are logged systematically, and fixes need to be applied and validated programmatically without human intervention. The FAIR Jupyter pipeline in particular currently stops at detecting and documenting failures — it does not explain them or repair them automatically. This thesis aims to close that specific gap: integrating an LLM-based repair layer directly into the pipeline so that dependency failures are explained and fixed autonomously, at scale, as part of the execution workflow itself. The approach additionally incorporates PyPI-grounded RAG to improve fix precision and enriches the FAIR Jupyter Knowledge Graph with structured repair provenance, thereby doubling the contribution: the system both repairs notebooks and extends the knowledge base with reproducibility outcomes.

4. Requirements and Objectives

This section defines the objectives and requirements of the proposed system. They are directly derived from the gap identified in Section 3: the FAIR Jupyter pipeline can detect dependency-related failures at scale, but cannot explain or repair them.

4.1 Objectives

O1 — Error Explanation. Develop a component that takes a dependency-related error message produced during notebook execution and generates a clear, human-readable explanation of what went wrong and why.

O2 — Automated Fix Generation. Develop a component that uses an open LLM, augmented with PyPI-grounded RAG, to automatically suggest and apply a fix for the detected dependency error, such as installing a missing package, pinning a specific version, or replacing a deprecated import.

O3 — Fix Validation. Re-execute the notebook after a fix has been applied and record whether the fix was successful. If the notebook still fails, the result should be logged with the updated error information for further analysis.

O4 — Benchmark Dataset. Compile a dataset of dependency-related notebook failures, applied fixes, and their outcomes. This dataset should be structured and reusable to support future research on automated notebook repair.

O5 — Pipeline Integration. Integrate all of the above components into the existing FAIR Jupyter pipeline so that the explanation and repair process runs automatically as part of the standard execution workflow, without requiring manual user intervention.

O6 — Knowledge Graph Enrichment. Map repair outcomes as structured RDF triples into the FAIR Jupyter Knowledge Graph, recording repair provenance including the original error, the applied fix, and the outcome. This extends the existing KG with actionable repair history.

4.2 Functional Requirements

FR1 — Dependency Error Detection. The system must be able to identify when a notebook execution failure is caused by a dependency-related error, such as a missing module, an incompatible package version, or a deprecated API.

FR2 — Error Explanation Generation. For each detected dependency failure, the system must generate a plain-language explanation of the error using an LLM, describing what the error means and what likely caused it.

FR3 — Fix Suggestion and Application. The system must use an LLM, augmented with live PyPI package version data retrieved via RAG, to suggest a concrete fix for the error and apply it automatically to the notebook environment or dependency configuration, without requiring user input.

FR4 — Re-execution and Validation. After applying a fix, the system must re-execute the notebook and determine whether the fix was successful, recording the outcome in a structured log.

FR5 — Result Recording. All outcomes including the original error, the generated explanation, the applied fix, and the re-execution result, must be recorded in a structured format (SQLite database) that can be used as a benchmark dataset.

FR6 — Pipeline Compatibility. The system must integrate with the existing FAIR Jupyter pipeline without requiring major changes to its architecture, extending it rather than replacing it.

FR7 — Knowledge Graph Integration. Repair outcomes must be serialized as RDF triples and inserted into the FAIR Jupyter Knowledge Graph, recording provenance information for each repair attempt.

4.3 Non-Functional Requirements

NFR1 — Use of Open Models. The system must rely exclusively on open-source, locally deployable LLMs. Proprietary models that require external API calls should not be used, to ensure the system is reproducible, cost-free to operate, and privacy-preserving.

NFR2 — Explainability. The error explanations generated by the system should be written in plain, non-technical language that is accessible to researchers who are not experienced programmers.

NFR3 — Reproducibility. The system itself must be reproducible. All components, model choices, prompts, and execution steps must be documented and version-controlled so that the entire pipeline can be re-run by others.

NFR4 — Scalability. The system should be able to process a large number of notebooks in batch without requiring manual intervention between executions.

NFR5 — Modularity. The LLM-based repair layer should be designed as a modular extension to the FAIR Jupyter pipeline, making it easy to swap out individual components, such as the choice of LLM or the prompting strategy without redesigning the whole system.

4.4 Out of Scope

To keep the thesis focused and achievable within the given time frame, the following are explicitly out of scope:

- Fixing errors that are not caused by dependency issues, such as logical errors in the notebook code, missing data files, or network-related failures.
- Supporting notebook languages other than Python.
- Guaranteeing that all dependency errors can be fixed. The system records failures as part of the benchmark dataset and does not aim for 100% repair coverage.

5. Proposed Approach

Before describing the steps of the proposed approach, it is important to define which error types this thesis targets. The focus is specifically on three categories: `ModuleNotFoundError`, when a required package is not installed; `ImportError`, when a package is present but cannot be imported correctly due to a version mismatch or a renamed module; and version incompatibility errors, when conflicting package requirements prevent the environment from being built. These were chosen because they are common in the FAIR Jupyter dataset and are realistically fixable through actions an LLM can suggest, such as installing a missing package or pinning a specific version. Errors such as `FileNotFoundError` or `TimeoutError` are excluded because they are caused by missing data files or hardware constraints, not dependency issues, and cannot be reasonably fixed based on error messages and dependency metadata alone.

The proposed system builds directly on top of the existing FAIR Jupyter pipeline, which already handles notebook execution, failure detection, dependency error filtering, and context extraction. The contribution of this thesis begins at step 5 in the workflow, once the pipeline has already identified a dependency-related failure and extracted the relevant context, including the error message, dependency specifications, and environment metadata. From that point, the system introduces five new steps.

Step 1 — LLM-based Explanation. The extracted context is passed to an open, locally deployable LLM with the task of generating a plain-language explanation of what caused the error and what it means. The goal is to make the failure understandable to users who may not have a deep programming background.

Step 2 — LLM-based Repair Suggestion with PyPI-grounded RAG. Using the same context, the system queries the PyPI registry to retrieve live package version information relevant to the failing dependency. This retrieved information is injected into the LLM prompt alongside the error context, grounding the repair suggestion in up-to-date package metadata. The LLM is then asked to suggest a concrete fix, such as installing a missing package at a specific version, pinning a compatible release, or replacing a deprecated import. This RAG-augmented approach is directly motivated by PLLM [5], which demonstrated that live retrieval of package data consistently improves fix rates over naive prompting.

Step 3 — Fix Application and Re-execution. The suggested fix is applied automatically to the notebook environment. The notebook is then re-executed to check whether the error has been resolved.

Step 4 — Result Storage. Regardless of whether the fix succeeded or failed, the full record — original error, explanation, suggested fix, and outcome — is stored in a SQLite database. This structured format makes the dataset queryable and reusable for future research.

Step 5 — Knowledge Graph Enrichment. The repair outcome is additionally serialized as structured RDF triples and inserted into the FAIR Jupyter Knowledge Graph. Each record

captures the notebook identifier, the error type, the applied fix, and the outcome, providing repair provenance that can be queried and reused by the broader research community. This step doubles the contribution of the system: beyond repairing notebooks, it enriches the knowledge graph with structured information about what was tried and what worked.

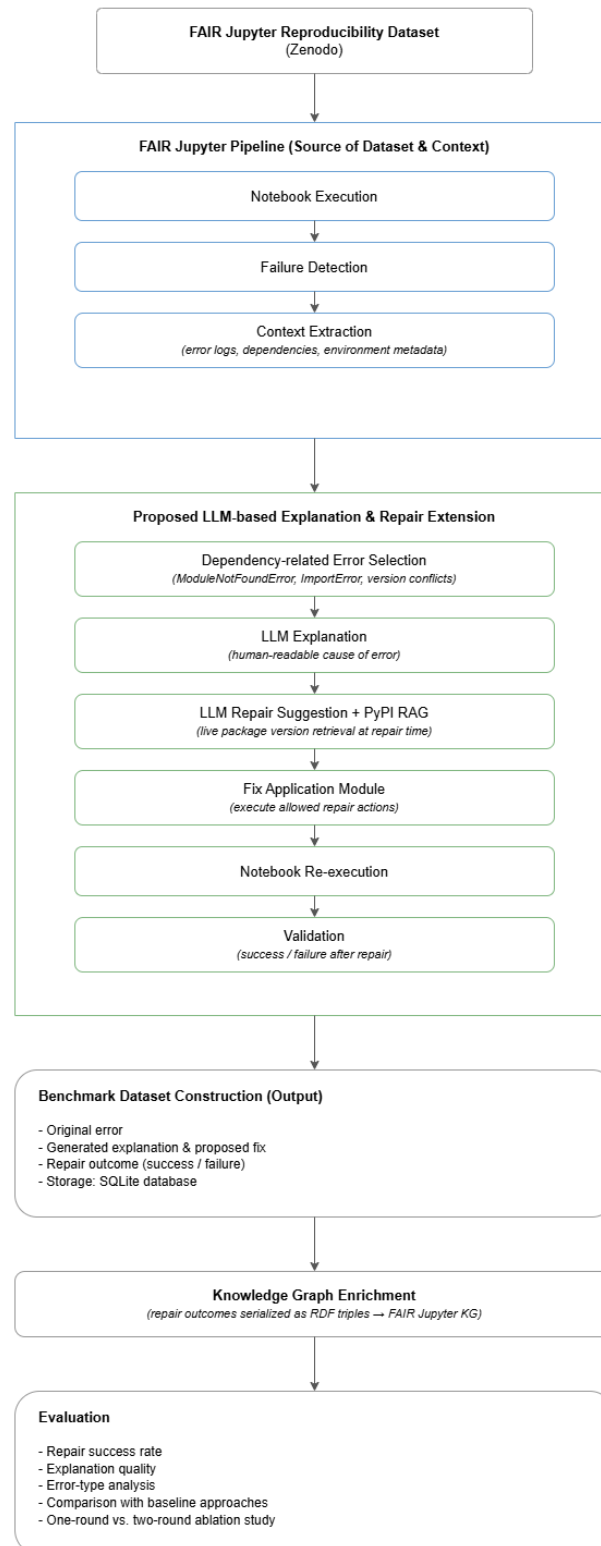


Figure 1. System architecture of the proposed approach

Legend

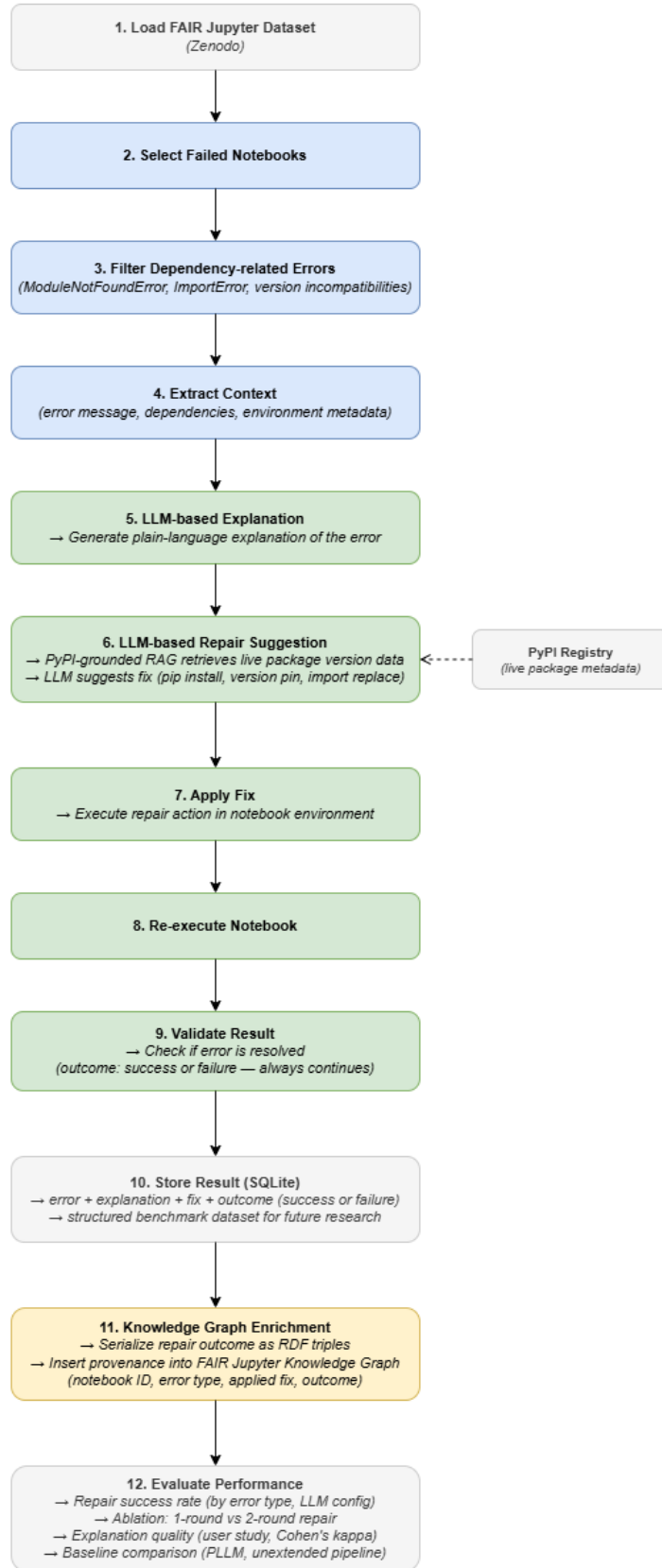
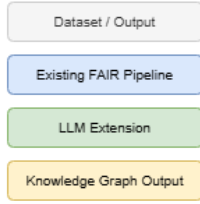


Figure 2. Workflow of the proposed LLM-based explanation and repair process

6. Evaluation Strategy

Evaluating this system involves two distinct aspects: measuring how well the LLM repairs dependency errors, and measuring how useful the generated explanations are. This section describes how each of these will be assessed.

6.1 Repair Success Rate

The primary metric for evaluating the repair component is the success rate — the proportion of notebooks that execute successfully after a fix has been applied. This is straightforward to measure automatically by checking the re-execution outcome recorded in step 3 of the proposed approach. Results will be broken down by error type (e.g., `ModuleNotFoundError`, `ImportError`, version incompatibilities) to understand which categories of dependency failures the system handles well and which it struggles with. The performance of different open LLMs and prompting strategies will also be compared to identify the best configuration for this specific task. An ablation study comparing one-round versus two-round repair will also be conducted to quantify the trade-off between additional computational cost and improvement in success rate, providing an empirical justification for the single-round design choice.

6.2 Explanation Quality

Evaluating the quality of generated explanations is less straightforward, as it involves a degree of subjectivity. The plan is to assess explanations along two dimensions: correctness — whether the explanation accurately describes the cause of the error and clarity — whether it is understandable to a non-expert user.

This will be done through a small user study involving participants recruited from computer science students and researchers at TU Chemnitz who have some programming experience but are not necessarily experts in dependency management. Each participant will be shown a sample of generated explanations alongside the original raw error message and asked to rate both using a 5-point Likert scale for each dimension. This allows a direct comparison between the raw technical error message and the LLM-generated explanation. To ensure the ratings are reliable, multiple participants will rate the same explanations and inter-rater agreement will be measured using Cohen's kappa. A kappa value above 0.6 will be considered acceptable agreement.

6.3 Benchmark Dataset

The benchmark dataset produced by the system containing errors, explanations, applied fixes, and outcomes, will itself be evaluated in terms of its size, coverage of different error types, and reusability for future research. The dataset will be made publicly available to support reproducibility and to contribute to the broader research community working on notebook repair and LLM-based program repair.

6.4 Baseline Comparison

Where possible, the system’s repair performance will be compared against existing approaches. The most natural baseline is the FAIR Jupyter pipeline without the LLM extension — comparing how many notebooks were previously unresolvable and how many the system can now fix. A comparison with PLLM [5] will also be included as a secondary baseline. Since PLLM is open source and its implementation is publicly available, it can be run on the same set of notebooks used in this thesis, making a direct comparison possible. This will help determine whether the proposed approach performs better or worse than an existing LLM-based dependency resolution system on the same task.

It is worth acknowledging that more rigorous repair benchmarks, such as ChatRepair [12], evaluate against a fixed set of known bugs with verified correct solutions, which allows for a precise measurement of repair accuracy. This thesis does not follow the same approach for the full dataset, because the notebooks come from real-world repositories where the correct dependency configuration is not always known in advance. However, for a smaller subset of notebooks where the correct environment can be confirmed, the system’s fixes will also be checked against that known correct state, not just by whether the notebook runs or not.

7. Conclusion

Jupyter notebooks play an important role in sharing and reproducing computational research, but dependency-related failures make a large proportion of published notebooks difficult or impossible to re-execute. The existing FAIR Jupyter pipeline has made significant progress in detecting and documenting these failures at scale, but does not yet attempt to explain or repair them. This thesis proposes to close that gap by integrating an open LLM-based repair layer directly into the pipeline, enabling it to automatically explain dependency errors in plain language, suggest and apply fixes using PyPI-grounded RAG, validate the results through re-execution, and enrich the FAIR Jupyter Knowledge Graph with structured repair provenance.

8. References

- [1] S. Samuel, D. Mietchen, H. Lo, and M. Gaedke, “Containing the Reproducibility Gap: Automated Repository-Level Containerization for Scholarly Jupyter Notebooks,” arXiv, arXiv:2604.01072
- [2] S. Samuel and D. Mietchen, “Computational Reproducibility of Jupyter Notebooks from Biomedical Publications,” *GigaScience*, vol. 13, p. giad113, 2024, doi: 10.1093/gigascience/giad113.
- [3] S. Samuel and D. Mietchen, “FAIR Jupyter: A Knowledge Graph Approach to Semantic Sharing and Granular Exploration of a Computational Notebook Reproducibility Dataset,” *Trans. Graph Data Knowl.* 2024.

- [4] J. F. Pimentel, L. Murta, V. Braganholo, and J. Freire, “A Large-Scale Study About Quality and Reproducibility of Jupyter Notebooks,” in Proc. 16th IEEE/ACM Int. Conf. Mining Software Repositories (MSR), 2019, doi: 10.1109/MSR.2019.00077.
- [5] A. Bartlett, C. Liem, and A. Panichella, “The Last Dependency Crusade: Solving Python Dependency Conflicts with LLMs,” arXiv, arXiv:2501.16191, Jan. 2025.
- [6] S. Samuel and B. König-Ries, “ReproduceMeGit: A Visualization Tool for Analyzing Reproducibility of Jupyter Notebooks,” arXiv, arXiv:2006.12110, Jun. 2020.
- [7] K. Grotov, S. Titov, Y. Zharov, and T. Bryksin, “Untangling Knots: Leveraging LLM for Error Resolution in Computational Notebooks,” arXiv, arXiv:2405.01559, Mar. 2024.
- [8] K. Grotov et al., “Debug Smarter, Not Harder: AI Agents for Error Resolution in Computational Notebooks,” arXiv, arXiv:2410.14393, Oct. 2024.
- [9] B. Szalontai et al., “Fine-Tuning CodeLlama to Fix Bugs,” in Proc. Int. Conf. Recent Innovations in Computing (ICRIC), Lecture Notes in Electrical Engineering, vol. 1195, Springer, 2024.
- [10] J. Wang, L. Li, and A. Zeller, “Restoring Execution Environments of Jupyter Notebooks,” in Proc. 43rd Int. Conf. Software Engineering (ICSE), 2021.
- [11] J. Wang, T.-Y. Kuo, L. Li, and A. Zeller, “Assessing and Restoring Reproducibility of Jupyter Notebooks,” in Proc. 35th IEEE/ACM Int. Conf. Automated Software Engineering (ASE), 2020.
- [12] C. S. Xia and L. Zhang, “Automated Program Repair via Conversation: Fixing 162 out of 337 Bugs for \$0.42 Each using ChatGPT,” in Proc. 33rd ACM SIGSOFT Int. Symp. Software Testing and Analysis (ISSTA), 2024, doi: 10.1145/3650212.3680323.
- [13] Jupyter AI, “Jupyter AI: A Generative AI Extension for JupyterLab,” GitHub, 2023. [Online]. Available: <https://github.com/jupyterlab/jupyter-ai>
- [14] F. Reyes et al., “Byam: Fixing Breaking Dependency Updates with Large Language Models,” arXiv, arXiv:2505.07522, May 2025.
- [15] B. Yang, Z. Cai, F. Liu, B. Le, L. Zhang, T. F. Bissyandé, Y. Liu, and H. Tian, “A Survey of LLM-based Automated Program Repair: Taxonomies, Design Paradigms, and Applications,” arXiv, arXiv:2506.23749, Jun. 2025.
- [16] X. Tang, J. Gao, J. Xu, T. Sun, Y. Song, S. Ezzini, W. C. Ouédraogo, J. Klein, and T. F. Bissyandé, “SynFix: Dependency-Aware Program Repair via RelationGraph Analysis,” in Findings of the ACL 2025, Vienna, Austria, Jul. 2025, pp. 4878–4894.